



Python程序设计与应用

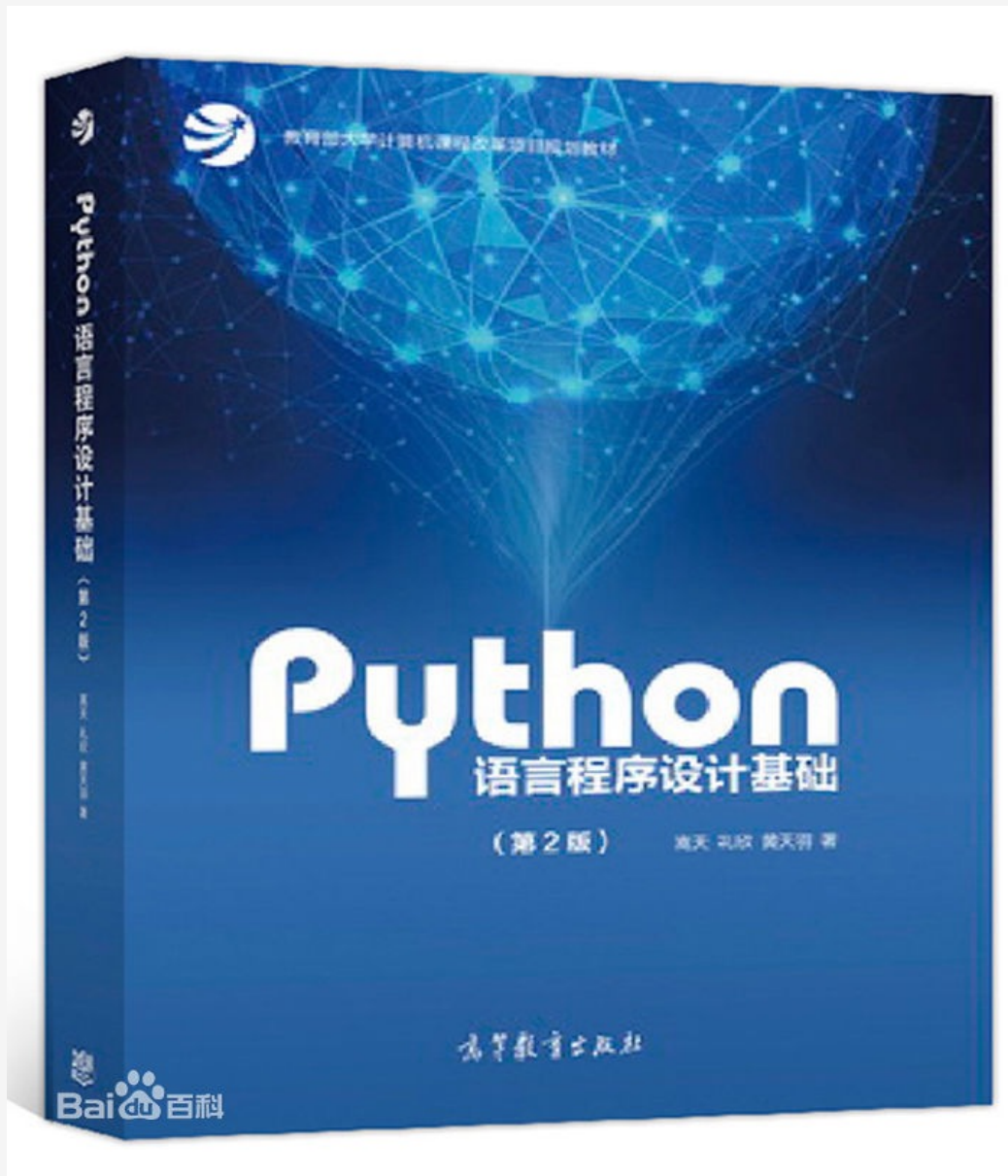
崔海

大连海事大学

cuihai@dlmu.edu.cn

个人简介

- 2013.09 – 2017.06 本科：大连理工大学软件学院软件工程专业
- 2017.09 – 2023.06 硕博：吉林大学计算机科学与技术学院计算机软件与理论专业
- 2023.12–2025.11 师资博士后：大连海事大学信息科学技术学院
- 2025.12–至今：讲师



Python 语言程序设计基础第2版

作者：高天 礼欣 黄天羽

出版：高等教育出版社



课程内容安排



教：课程将讲授哪些内容？ 教学内容如何安排？



学：如何学习课程内容？ 需要多少学习时间？



效：如何通过这门课程？ 学完取得什么效果？



教：课程内容介绍

- 程序设计那些事儿（第1-2周）
- 枯燥的Python语言语法（第3-8周）
- 高级程序设计方法（第9-10周）
- 看起来有点趣味的Python应用（第11-15周）
- 梳理梳理我们都学了什么（第16周）



教：课程内容介绍

■ 程序设计那些事儿（第1-2周）

计算机概念

程序设计语言

编译和解释

Python历史和发展

Python的优势特性

IPO基本程序设计方法

Python语言开发环境配置

Python版本迭代

目标：编写Hello World，运行10行左右小程序

教：课程内容介绍

■ 枯燥的Python语言语法（第3-7周）

数字类型

字符串类型

元组类型

列表类型

分支结构

循环结构

函数

递归

字典类型

文件操作

库函数使用

Python语法全貌

目标：编写50行左右有用程序

教：课程内容介绍

■ 高级程序设计方法（第8-9周）

计算思维

Python程序调试

自顶向下和自底向上设计

Python第三方库的安装和使用

目标：编写100行左右有用程序

■ 看起来有点趣味的Python应用（第10-15周）

网络爬虫和信息提取

数据分析与展示

目标：初步使用程序设计解决一般计算问题

■ 梳理梳理我们都学了什么（第16周）

.....

目标：回归现实，顺利通过考试！



学：两点学习建议

- 紧跟进度：跟随课程进度，完成课程要求的学习内容
- 勤加练习：除了安排的上机，适当进行额外的程序练习



效：考核和成绩

■ 平时成绩(30分)

- 基础分：15分
- 出勤 & 小测：5 次，共 15 分

■ 期末成绩(70分)

- 题型(百分制)：选择(20分)、填空(20分)、判断(20分)、编程(40分)



python入门

01

PART ONE

计算机的概念

The Concept of Computer

- 计算机是根据指令操作数据的设备
- 具备**功能性**和**可编程性**两个基本特性
 - 功能性指对数据的操作，表现为数据计算、输入输出处理和结果存储等
 - 可编程性指它可以根据一系列指令自动地、可预测地、准确地完成操作者的意图



计算机的概念

- 只要设备具备了计算的功能性和操作的可编程性就可以看作是计算机
- 判断一个设备是否属于计算机并不依靠其制造材质，计算机不一定是电子的
 - 光计算机、量子计算机、超导计算机、生物计算机等新形态计算机都不是建立在电子学基础上，但它们都表达了计算机的概念，也属于计算机类别

除特殊说明，本课程后续内容中的“计算机”均指电子计算机

- 计算机技术发展主要围绕计算机的功能性和可编程性展开
 - 计算机硬件所依赖的集成电路按照**摩尔定律**以指数方式增长
 - 计算机运行速度也接近几何级数快速增加
 - 计算机所能高效支撑的功能不断丰富发展
 - 表达计算机可编程性的设计语言也在经历从机器语言、汇编语言到高级语言的发展过程，并逐步朝着更接近自然语言的方向发展

■ 摩尔定律是计算机发展历史上最重要的预测法则

- 由英特尔(Intel)创始人之一戈登·摩尔(Gordon Moore)在1965年提出
- 其内容为：当价格不变时，集成电路上可容纳的元器件的数目，约每隔18-24个月便会增加一倍，性能也将提升一倍。换言之，每一美元买到的电脑性能，将每隔18-24个月翻一倍以上
- 这一定律揭示了信息技术进步的速度，成为计算机性能水平的一个重要预测法则

Our World
in Data

Transistor count



■ 华为韬（ τ ）定律

- 2026年5月26日，国际电路与系统研讨会上，华为公司董事、半导体业务部总裁何庭波发表了题为《半导体新路径探索与实践》的主旨演讲，正式提出韬（ τ ）定律



■ 华为韬（ τ ）定律

➤ 什么是“韬（ τ ）”？

- 韬（ τ ），是时间常数的符号。它是物理学和工程学中的一个重要参数，用于描述系统对输入信号或扰动的响应速度

➤ 什么是“韬（ τ ）定律”？

- 以“时间缩微”替代“几何缩微”，以系统性降低时间常数（ τ ）为目标，通过逻辑折叠等创新技术，持续压缩信号传播时延，不断提升晶体管密度，实现半导体与电子系统的持续演进

韬(τ)定律 VS 摩尔定律

—— 两条芯片发展的不同道路 ——

τ	韬(τ)定律	对比维度	摩尔定律
	2026年 (ISCAS 2026提出)	提出时间	1965年
	华为何庭波团队	提出者	戈登·摩尔 (Gordon Moore)
	时间缩微 (Temporal Scaling) 降低系统时间常数 τ	核心思想	几何缩微 (Geometric Scaling) 缩小晶体管尺寸
	信号传输越来越快 系统延迟越来越低	发展路径	晶体管越来越小 单位面积集成度越来越高 
	逻辑折叠 + 3D堆叠 + 高密度互连 (从二维走向三维)	技术方向	制程微缩 (2D平面缩小) (180nm $\rightarrow$... $\rightarrow$ 7nm $\rightarrow$ 5nm $\rightarrow$ 3nm)
	先进封装、系统架构设计、 材料与互连技术	关键依赖	EUV光刻机, 先进制程工艺、 晶圆制造能力 
	仍处于验证阶段, 生态正在建设中	面临的问题	成本暴涨、功耗升高、良率下降, 逼近物理极限
	降低系统延迟, 提升整体效率 实现性能持续提升	最终目标	提高晶体管密度, 提升芯片性能 (性能 $\propto$ 晶体管数量) 

一句话理解

韬(τ)定律
让数据跑得更近

- ✓ 3D堆叠
- ✓ 逻辑折叠
- ✓ 近距离互连
- ✓ 更低延迟

提升性能



VS

摩尔定律
把芯片做得更小

- ✓ 制程微缩
- ✓ 晶体管变小
- ✓ 密度提升
- ✓ 性能提升

提升性能





计算机技术发展水平的时代性

■ 第一阶段：计算机系统结构阶段 (1946年-1981年)

- 以全球首台数字计算机ENIAC诞生为标志
- 主要围绕计算机系统结构设计展开
- 随着 IBM PC (型号为 IBM 5150) 为代表的个人计算机的诞生，计算机技术进入面向大众的新阶段

■ 第二阶段：计算机网络和视窗阶段(1982年-2007年)

- 以面向全球子网间组网的TCP/IP网络协议为标志
- 主要围绕网络技术、视窗技术、多媒体技术展开
- 诞生了具有跨平台功能的Java语言(1995年)
- 随着iPhone智能手机的推出和广泛普及，计算机技术进入面向移动网络应用的新阶段



计算机技术发展水平的时代性

■ 第三阶段：复杂信息系统阶段(2008年-2010s中期)

- 以安卓(Android)开源移动操作系统的发布为起点
- 出现移动互联网、云计算、可信计算、大数据、可穿戴计算、物联网、互联网+等概念
- python语言(2008年3.0版本)已经成为这个阶段计算机系统的主流编程语言

■ 第四阶段：人工智能阶段(2010s末-至今)

- 计算机技术从处理信息、提供服务，进阶到能够感知环境、学习规律、辅助决策甚至自主创造
- 大语言模型与生成式AI：以ChatGPT（2022年底发布）为代表的生成式AI展现出惊人的内容生成、逻辑推理和对话能力，python成为主导语言

PART TWO

程序设计语言

Programming Language

- **程序设计语言**：也叫**编程语言**，是计算机能够理解和识别用户操作意图的一种交互体系
 - 按照特定规则组织计算机指令，使计算机能够自动进行各种运算处理
 - 按照程序设计语言规则组织起来的一组计算机指令称为计算机程序

■ 程序设计语言包括：机器语言、汇编语言、高级语言

➤ 机器语言：二进制语言，直接使用二进制代码表达指令

- ✓ 是计算机硬件可以直接识别和执行的程序设计语言
- ✓ 程序员将用0,1数字编程的程序代码打在纸带或卡片上，1打孔，0不打孔，再将程序通过纸带机或卡片机输入计算机，进行运算
- ✓ 直接使用机器语言编写程序十分繁冗，难以阅读和修改

执行数字2和3的加法

16位计算机上的机器指令：11010010 00111011

■ 程序设计语言包括：机器语言、汇编语言、高级语言

➤ 汇编语言：是二进制指令的文本形式，使用助记符与机器语言中的指令进行一一对应

- ✓ 相对于机器指令更容易记住
- ✓ 在不同的设备中，汇编语言对应着不同的机器语言指令集 (台式机 / 笔记本 (Intel、AMD) 使用 x86；手机 / 平板 / Apple M 系列适用 ARM)
- ✓ 程序员用汇编语言写出源代码，再用汇编器将其编译为机器码，最后由计算机执行

■ 程序设计语言包括：机器语言、汇编语言、高级语言

➤ 汇编语言：汇编指令按功能可分为四类（以 x86 为例）

- 数据传送：MOV（传送）、PUSH/POP（压栈/出栈）、XCHG（交换）
- 算术运算：ADD（加）、SUB（减）、INC/DEC（加一/减一）、MUL/DIV（乘/除）
- 逻辑运算：AND、OR、XOR、NOT、SHL/SHR（移位）
- 控制转移：CMP（比较）、JMP（无条件跳转）、JE/JNE（条件跳转）、CALL/RET（调用/返回）

程序设计语言发展

■ 程序设计语言包括：机器语言、汇编语言、高级语言

➤ 汇编语言：实例计算 $2 + 3$

```
mov ax, 2      ; 把数字 2 放进寄存器 AX
mov bx, 3      ; 把数字 3 放进寄存器 BX
add ax, bx     ;  $AX \leftarrow AX + BX$ , 结果 5 留在 AX 中
```

- 寄存器是 CPU 内部的微型存储单元，容量极小，但读写速度远快于内存
- CPU 的运算只能在寄存器上进行，数据必须先从内存"搬进"寄存器，算完再"搬回"内存
- x86 的 16 位通用寄存器有 AX、BX、CX、DX 四个

程序设计语言发展

■ 程序设计语言包括：机器语言、汇编语言、高级语言

➤ 高级语言：是接近自然语言的一种计算机程序设计语言，可以更容易地描述计算问题并利用计算机解决计算问题

- ✓ C、C++、Java、Python 等高度封装的编程语言
- ✓ 具有更强的表达能力，且与计算机的硬件结构和指令系统无关，同一编程语言在不同计算机上的表达方式是一致的
- ✓ 汇编语言和高级语言最后都需要转换成机器语言

程序设计语言经历了从机器语言到高级语言的发展过程，朝着更接近自然语言的方向发展！

编译和解释

这里所说的执行方式指计算机执行一个程序的过程，静态语言采用**编译**执行，脚本语言采用**解释**执行

■ 高级语言按照计算机执行方式的不同可分成两类

➤ 静态语言： C++/Java/C#

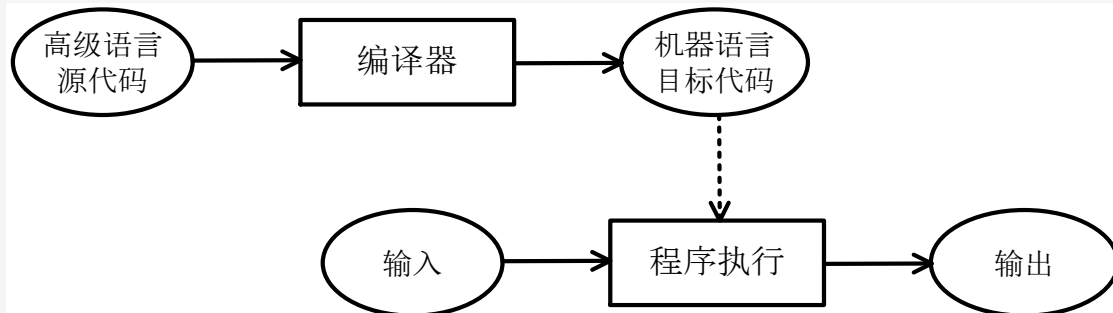
➤ 脚本语言： Python/PHP/ASP/Ruby/Perl/

■ 编译是将源代码转换成目标代码的过程

➤ 通常源代码是高级语言代码，目标代码是机器语言代码

➤ 执行编译的计算机程序称为**编译器**(Compiler)

➤ 编译器将源代码转换成目标代码，计算机可以**立即或稍后**运行这个目标代码



■ 编译过程示例

- 词法分析。将 “int c = 2 + 3;” 字符流切成一个个“词” (Token)

Token	类别
int	关键字
c	标识符
=	赋值运算符
2	整型字面量
+	运算符
3	整型字面量
;	分隔符

c

```
int main() {  
    int c = 2 + 3;  
    return 0;  
}
```

■ 编译过程示例

- 语法分析。依据语言的文法规则，把 Token 序列组织成*抽象语法树*（AST），确定运算的结合与优先关系

声明语句

└─ 类型: int

└─ 变量: c

└─ 初始值: 加法表达式

└─ 左: 2

└─ 右: 3

c

```
int main() {  
    int c = 2 + 3;  
    return 0;  
}
```

■ 编译过程示例

- 语义分析。检查“合乎语法但不合乎道理”的问题。2 和 3 都是整数，能相加，类型没问题；如果写成 `2 + "你好"` 在这一步报错
- 汇编指令生成。输出汇编代码

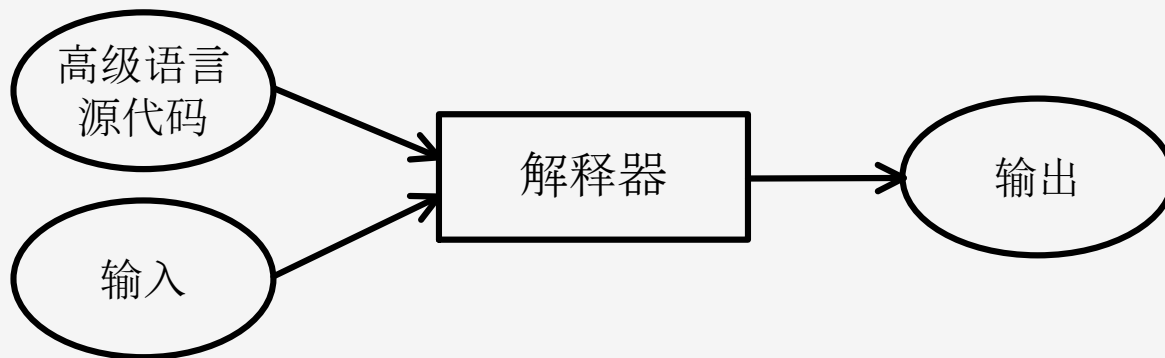
```
mov ax, 2      ; 把 2 放进寄存器
add ax, 3      ; 加上 3, 结果 5
mov [c], ax    ; 存到变量 c
```

```
c
int main() {
    int c = 2 + 3;
    return 0;
}
```

- 转成机器码。10111000..., 链接后打包成 .exe 可执行文件

- 解释是将源代码逐条转换成目标代码同时逐条运行目标代码的过程

- 执行解释的计算机程序称为**解释器**(Interpreter)
- 高级语言代码与数据一同输入给解释器，然后输出运行结果



- 解释过程示例: 以 $c = 2 + 3$ 为例 (Python)
 - 词法分析。将字符流切成一个个"词" (Token)

Token	类别
c	标识符
=	赋值运算符
2	整型字面量
+	运算符
3	整型字面量

- 解释过程示例: 以 $c = 2 + 3$ 为例 (Python)
 - 语法分析。按文法规则构建*抽象语法树* (AST)

赋值语句

└─ 目标: c

└─ 值: 加法表达式

└─ 左: 2

└─ 右: 3

⚠ 到这一步为止, 与编译器的处理完全一致

■ 解释过程示例: 以 $c = 2 + 3$ 为例 (Python)

- 编译为字节码。遍历*抽象语法树* (AST) 生成 **Python 字节码**——一种面向栈式虚拟机的中间指令，与具体 CPU 架构无关

```
LOAD_CONST    2        # 把常量 2 压入运算栈
LOAD_CONST    3        # 把常量 3 压入运算栈
BINARY_ADD    # 弹出栈顶两个数，相加，结果压回栈
STORE_NAME    c        # 弹出栈顶，绑定到名字 c
```

⚠ Python 会把字节码缓存为 .pyc 文件（存于 `__pycache__` 目录）

■ 解释过程示例: 以 $c = 2 + 3$ 为例 (Python)

- 虚拟机逐条执行。字节码交给 *Python 虚拟机* (*PVM*)。虚拟机不断重复“取指—译码—执行”

取出一条字节码指令



判断它是哪条指令 (LOAD_CONST? BINARY_ADD?)



跳转到对应的 C 函数, 完成该操作



指令指针后移, 取下一条

- 输出结果, 继续读取下一行。上一行执行完毕, 虚拟机继续处理后续语句



编译和解释

阶段	编译型 (C/C++)	解释型 (Python)
词法分析	✓	✓
语法分析 (建 AST)	✓	✓
语义分析、类型检查	编译时完成	推迟到运行时逐次进行
中间代码	三地址码 / LLVM IR	Python 字节码
优化	大量、深入	有限 (时间预算紧张)
最终产物	机器码, 存为 <code>.exe</code>	字节码, 缓存为 <code>.pyc</code>
谁来执行	CPU 硬件直接执行	虚拟机软件逐条模拟执行
执行速度	快	慢 (约数十倍)
跨平台	需为每个平台重新编译	换个解释器即可
错误暴露时机	编译期即报错	运行到该行才报错

03

PART THREE

为什么是Python ?

Why study Python?



Python : 读音 /'paɪθən/

Python的创始人为吉多·范罗苏姆 (Guido van Rossum, 龟叔)。1989年圣诞节期间，在阿姆斯特丹，Guido为了打发圣诞节的无趣，决心开发一个新的脚本解释程序，可以兼具C和Shell的优点。之所以选中Python（大蟒蛇的意思）作为该编程语言的名字，是因为他是一个叫Monty Python的喜剧团体的爱好者。



Python发展历程

1989年，Guido开始着手设计开发Python，1991年初发布了第一个公开发行人版



2000年，Python发布2.X版本，开启了Python广泛应用的新时代
2010年，Python2.X系列发布最后一个版本2.7



2008年，Python发布3.X版本

3.x版本在语法层面和解释器内部做了很多重大改进，无法向下兼容python 2.0系列的既有语法

Python 2.x版本已经是遗产，Python 3.x是这个语言的现在和未来



Python在编程语言排行榜位置

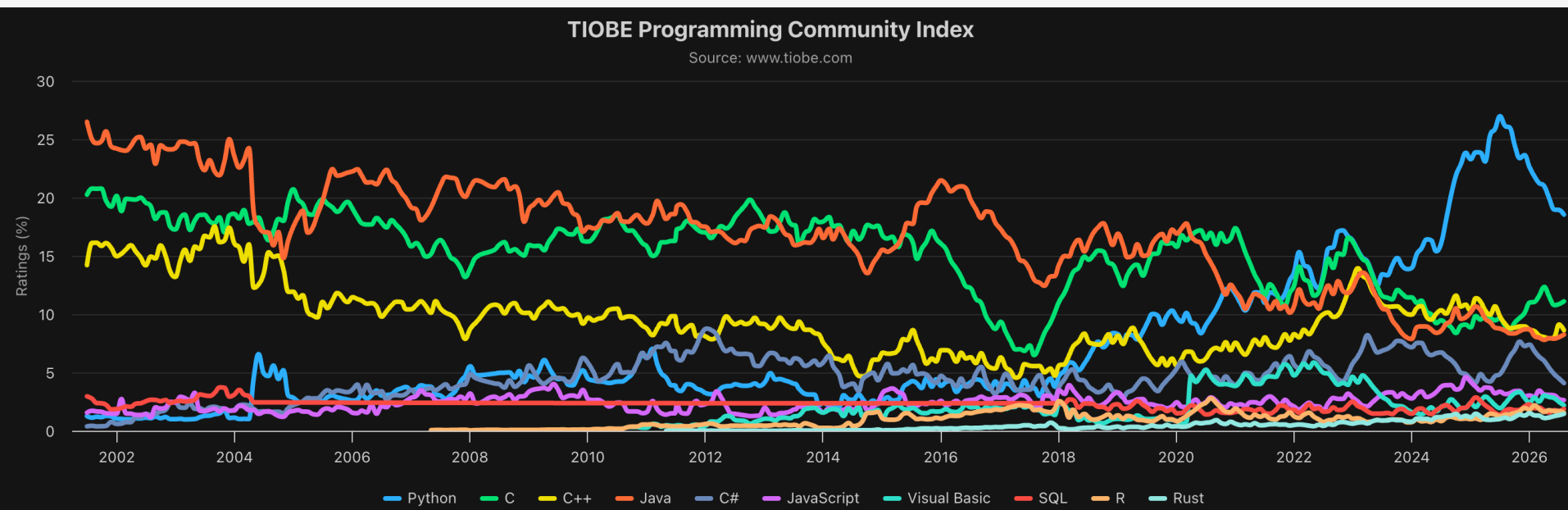
TIOBE 排行榜 (<https://www.tiobe.com/tiobe-index/>)

Aug 2026	Aug 2025	Change	Programming Language		Ratings	Change
1	1			Python	18.53%	-7.61%
2	3	▲		C	11.10%	+2.07%
3	2	▼		C++	8.62%	-0.56%
4	4			Java	8.25%	-0.34%
5	5			C#	4.09%	-1.43%
6	6			JavaScript	2.63%	-0.52%
7	7			Visual Basic	2.18%	-0.15%
8	12	▲▲		SQL	1.88%	+0.16%
9	14	▲▲		R	1.56%	+0.19%
10	18	▲▲		Rust	1.45%	+0.31%



Python在编程语言排行榜位置

TIOBE 排行榜 (<https://www.tiobe.com/tiobe-index/>)



Python语言的特点

- **语法简洁**：实现相同功能，Python语言的代码行数仅相当于其他语言的1/10~1/15

```
>>> print("Hello World")
Hello World
>>>
```

第一行的 “>>>” 是Python语言运行环境的提示符

第二行是Python语句的执行结果

```
#include <stdio.h>
int main(void)
{
    printf("Hello World\n");
    return 0;
}
```

C语言程序除了向屏幕输出 “Hello World” 的printf语句外，还包含了include、int、main、printf、return等其他辅助元素

更少的代码行数、更简洁的表达方式将带来更少的程序错误、更快的程序开发速度和更好的可读性



Python语言的特点

- “Hello, World”程序指的是只在计算机屏幕上输出“Hello, World!”（意为“世界，你好！”）这行字符串的计算机程序
- 一般来说，这是每一种计算机编程语言中最基本、最简单的程序，亦通常是初学者所编写的第一个程序
- 它还可以用来确定该语言的编译器、程序开发环境，以及运行环境是否已经安装妥当



Python语言的特点

- **与平台无关**：可以在任何安装解释器的计算机环境中执行，可不经修改地实现跨平台运行
- **粘性扩展**：可以集成C、C++、Java等语言编写的代码，通过接口和函数库等方式将它们整合在一起
- **开源且类库丰富**：<http://pypi.python.org/>，截止到2026年8月，已收录超过 88 万个第三方开源库



Python语言的特点

- **支持中文**：Python 3.0解释器采用 **UTF-8 编码**表达所有字符信息，可表达英文、中文、韩文、法文等各类语言，因此在处理中文时更加灵活且高效
 - ASCII编码。由于计算机是美国人发明的，最早只有127个字符被编码到计算机里，也就是大小写英文字母、数字和一些符号。比如大写字母 A 的编码是65，小写字母 z 的编码是122
 - GB2312编码。收录进中文
 - Unicode编码。把所有语言都统一到一套编码里，防止乱码出现，最常用的是UCS-16编码，用两个字节表示一个字符（非常偏僻的字符除外）
 - UTF-8编码。将Unicode编码转化为“可变长编码”。常用的英文字母被编码成1个字节，汉字通常是3个字节

Python语言的特点

- **强制可读**：Python通过强制**缩进**来体现语句间的逻辑关系，显著提高了程序的可读性，进而增加了Python程序的可维护性

- C语言与 Python 的比较

- C语言靠花括号 {} 划分代码块，缩进只是习惯

```
if (score >= 60) {  
    printf("及格\n");  
    printf("恭喜\n");  
}
```



```
if (score >= 60) { printf("及格\n"); printf("恭喜\n"); }
```



Python语言的特点

- **强制可读**：Python通过强制**缩进**来体现语句间的逻辑关系，显著提高了程序的可读性，进而增加了Python程序的可维护性

- C语言与 Python 的比较

- Python 靠缩进划分代码块，缩进就是语法

```
if score >= 60:
```

```
    print("及格")
```

```
    print("恭喜")
```



- 缩进错了直接报错，程序根本无法运行



```
if score >= 60:
```

```
print("及格")
```

```
# IndentationError: 缺少缩进
```



Python语言的特点

- **模式多样**：Python 3.0解释器内部采用面向对象方式实现，但语法层面同时支持面向对象和面向过程两种编程方式，这为使用者提供了灵活的编程模式
 - 面向过程：把任务拆成一步步的函数调用

```
def area(w, h):  
    return w * h
```

```
w1, h1 = 3, 4  
print(area(w1, h1))      # 输出 12
```



Python语言的特点

- **模式多样**：Python 3.0解释器内部采用面向对象方式实现，但语法层面同时支持面向对象和面向过程两种编程方式，这为使用者提供了灵活的编程模式

➤ 面向对象：把数据和操作打包成"对象"

数据（self.w、self.h）
与操作（area 方法）
封装在一起

```
class Rectangle:
    def __init__(self, w, h):
        self.w = w
        self.h = h

    def area(self):
        return self.w * self.h
```

```
r = Rectangle(3, 4)
print(r.area())          # 输出 12
```



Python语言的特点

- **模式多样**：Python 3.0解释器内部采用面向对象方式实现，但语法层面同时支持面向对象和面向过程两种编程方式，这为使用者提供了灵活的编程模式
 - ⚠ **Java** 强制一切代码必须写在类里，哪怕只是打印一行"Hello World"也要先定义一个类；**C** 语言则完全没有类的概念。**Python** 处在两者之间

java

```
public class Hello {  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

c

```
#include <stdio.h>  
  
int main() {  
    printf("Hello World\n");  
    return 0;  
}
```

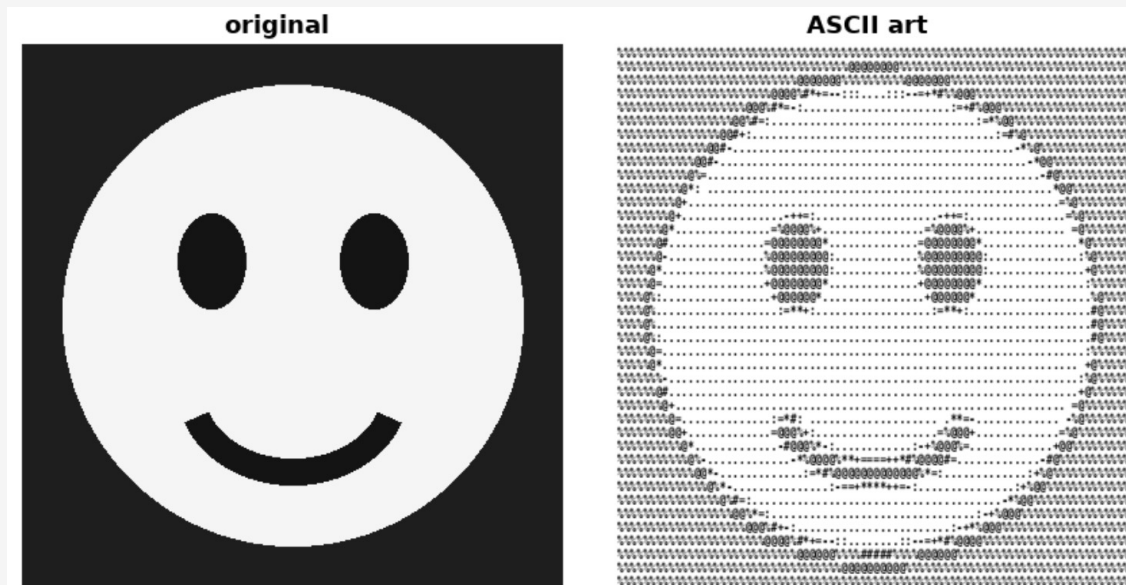
Python可以干什么

 Python	+	 Pandas	→	 Data Manipulation Clean, transform and analyze data easily.
 Python	+	 Scikit-learn	→	 Machine Learning Build and evaluate ML models.
 Python	+	 TensorFlow	→	 Deep Learning Build and train deep neural networks.
 Python	+	 Matplotlib	→	 Data Visualization Create static, animated, and interactive plots.
 Python	+	 Seaborn	→	 Advanced Charts Statistical data visualization made easy.
 Python	+	 BeautifulSoup	→	 Web Scraping Extract data from websites.
 Python	+	 Selenium	→	 Browser Automation Automate tasks in the browser.
 Python	+	 FastAPI	→	 High-performance APIs Build fast, modern and efficient APIs.
 Python	+	 SQLAlchemy	→	 Database Access Interact with databases using Python.
 Python	+	 Flask	→	 Lightweight Web Apps Build simple and powerful web apps.
 Python	+	 Django	→	 Scalable Platforms Build secure, scalable web platforms.
 Python	+	 OpenCV	→	 Computer Vision Process and analyze images and videos.
 Python	+	 Pygame	→	 Game Development Create 2D games and interactive apps.

Python的缺点

■ 运行速度慢

- 和C语言程序相比非常慢 (Python是解释型语言)
- 图片变成 ASCII 字符画示例
 - 把一张照片变成用键盘字符拼出来的画
 - 图片上暗的地方用笔画密的字符（比如 @、#），亮的地方用笔画稀的字符（比如 .、空格）



Python的缺点

- 图片变成 ASCII 字符画示例



■ 图片变成 ASCII 字符画示例

➤ 处理步骤

- 读入图片。把 4000×3000 的照片读进内存。每个像素存三个字节（R、G、B 各一个），总共 $1200 \text{ 万} \times 3 = 3600 \text{ 万字节}$
- 计算灰度。每个像素按照RGB加权平均，将彩色图片变成黑白
- 分块取平均。所以把原图切成 200×100 个方块，每块 $20 \times 30 = 600$ 个像素，取平均值作为这一块的整体亮度
- 亮度换字符。准备一个由密到疏的字符表 @%#*+ = -.:，共 10 个。按亮度查表。亮度 $0 \sim 25 \rightarrow @$ （笔画最密，看着最黑）；亮度 $230 \sim 255 \rightarrow$ 空格（最亮）



Python的缺点

■ 图片变成 ASCII 字符画示例

➤ C语言代码

```
#include <stdio.h>
#include <time.h>
#define CW 200
#define CH 100

static unsigned char rgb[3000*4000*3];
static double sum[CH][CW]; static int cnt[CH][CW];

int main() {
    char t[3]; int w, h, m;
    FILE *f = fopen("ship.ppm", "rb");
    fscanf(f, "%2s %d %d %d", t, &w, &h, &m); fgetc(f);
    fread(rgb, 1, (size_t)w*h*3, f);

    clock_t s = clock();
    for (int y = 0; y < h; y++)
        for (int x = 0; x < w; x++) {
            long i = ((long)y*w + x) * 3;
            sum[y*CH/h][x*CW/w] += 0.299*rgb[i] + 0.587*rgb[i+1] + 0.114*rgb[i+2];
            cnt[y*CH/h][x*CW/w] += 1;
        }
    printf("%.3f s\n", (double)(clock()-s)/CLOCKS_PER_SEC);

    const char *C = "@%#*+=-:. ";
    FILE *o = fopen("art.txt", "w");
    for (int r = 0; r < CH; r++, fputc('\n', o))
        for (int c = 0; c < CW; c++)
            fputc(C[(int)(sum[r][c]/cnt[r][c]) * 9 / 255], o);
}
```

耗时 0.03s



Python的缺点

■ 图片变成 ASCII 字符画示例

➤ Python语言代码

```
from PIL import Image
import time

CW, CH = 200, 100
img = Image.open("ship.ppm"); w, h = img.size; px = img.load()

s = time.time()
total = [[0.0]*CW for _ in range(CH)]
count = [[0]*CW for _ in range(CH)]
for y in range(h):
    for x in range(w):
        r, g, b = px[x, y]
        total[y*CH//h][x*CW//w] += 0.299*r + 0.587*g + 0.114*b
        count[y*CH//h][x*CW//w] += 1
print("%.3f s" % (time.time() - s))

CHARS = "@%#*+=-:. "
open("art.txt", "w").write("\n".join(
    "".join(CHARS[int(total[r][c]/count[r][c])*9//255] for c in range(CW))
    for r in range(CH)))
```

耗时 4.91s
(慢 164 倍)

[illegible]



Python的缺点

■ 代码不能加密

- 若要发布Python程序，实际上就是发布源代码，
- 与C语言不同，C语言不用发布源代码，只需要把编译后的机器码（也就是你在Windows上常见的xxx.exe文件）发布出去

04

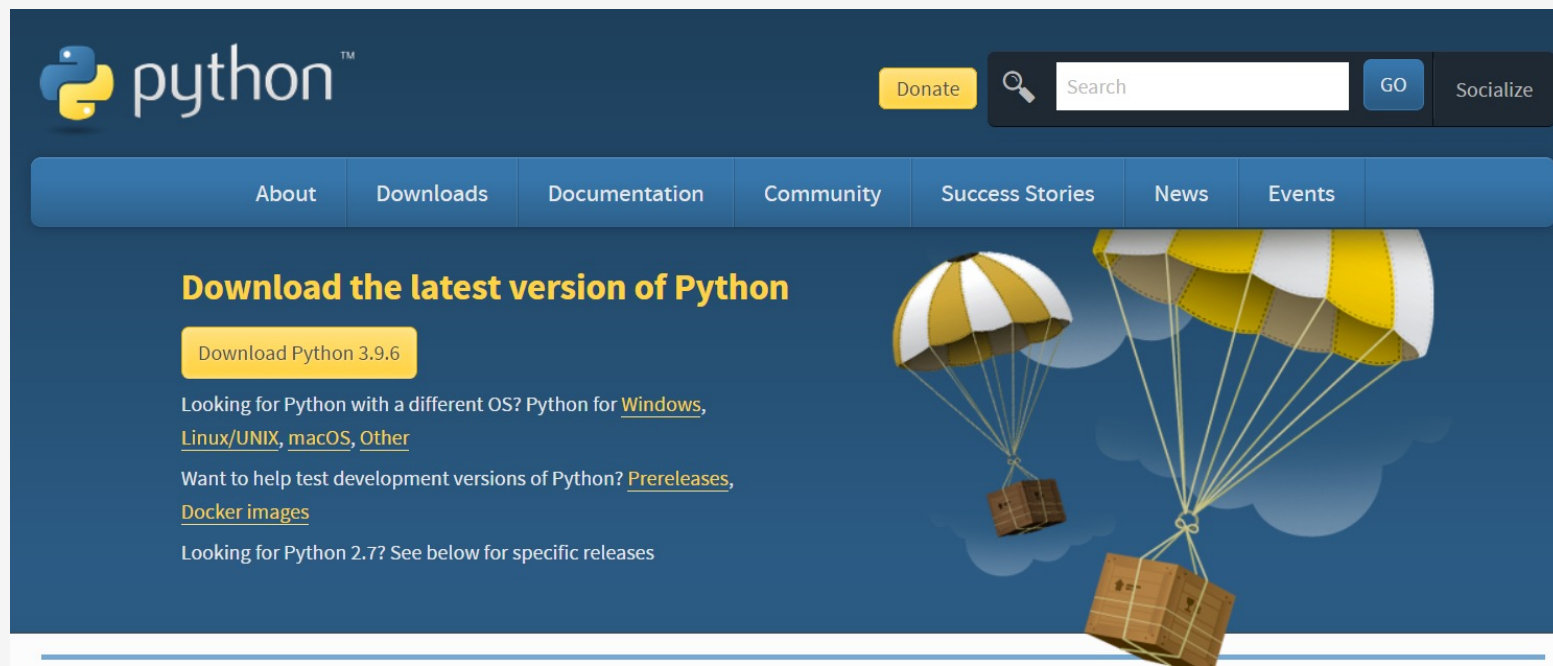
PART FOUR

Python开发环境配置

Configuration of Python

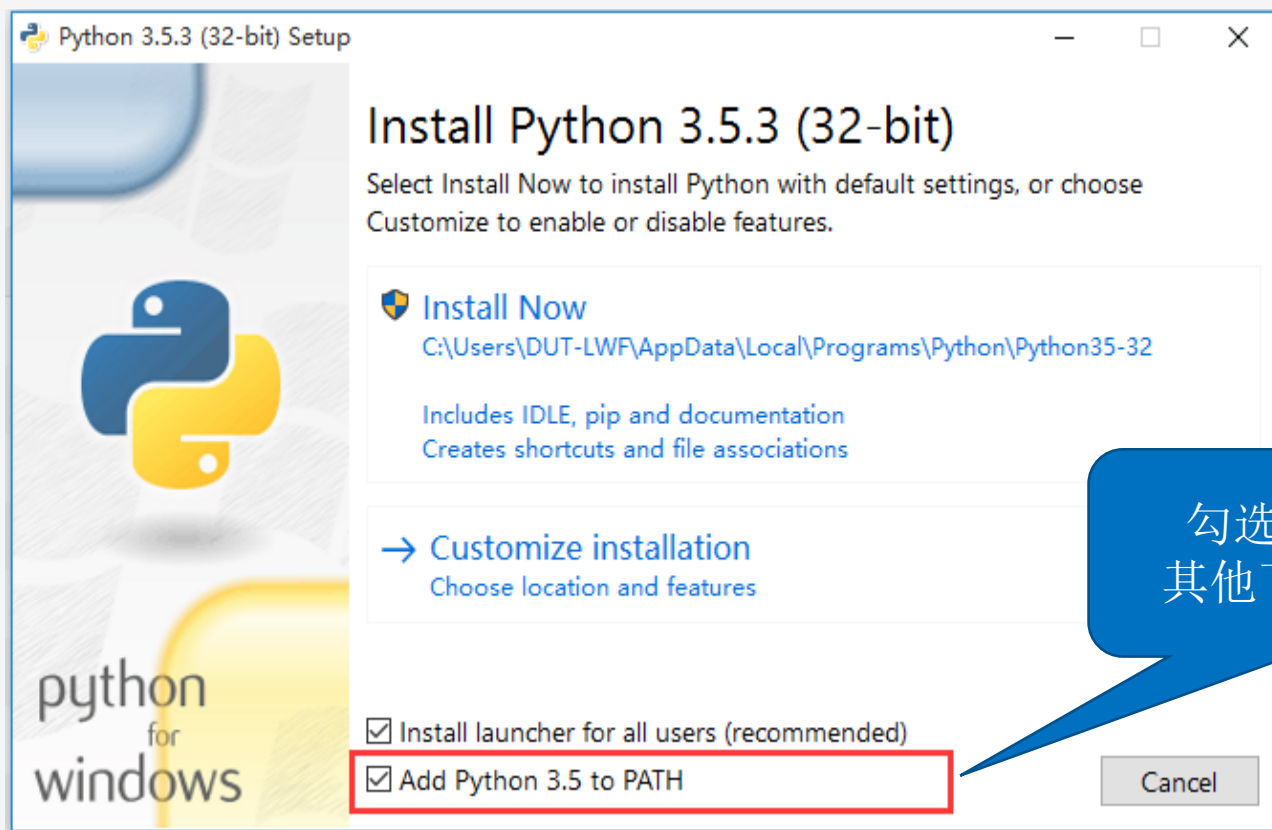
Python下载

- 到Python主页下载并安装Python基本开发和运行环境
<https://www.python.org/downloads/>
- 根据操作系统不同选择不同操作版本
- 下载相应的Python 3.0系列版本程序



Python安装

- 双击下载的程序安装Python解释器，将启动下图所示的引导过程



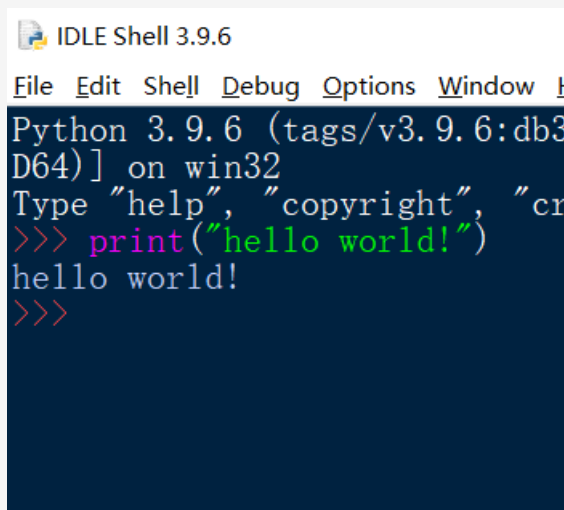
勾选该复选框
其他下一步即可

- Python安装包将在系统中安装一批与Python开发和运行相关的程序，最重要的两个是Python命令行和Python集成开发环境IDLE

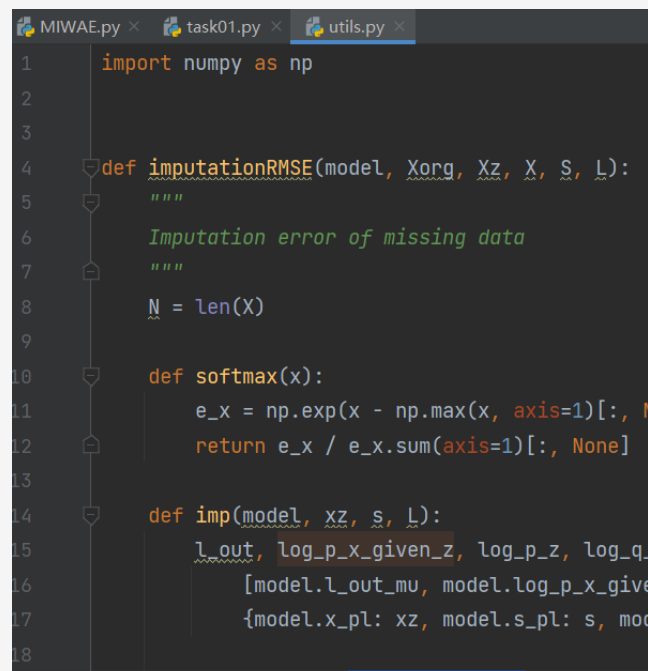
Python运行

■ 有两种方式：交互式和文件式

- **交互式**：指Python解释器即时响应用户输入的每条代码，给出输出结果，一般用于调试少量代码
- **文件式**：也称批量式，指用户将Python程序写在一个或多个文件中，然后启动Python解释器批量执行文件中的代码，是最常用的编程方式



```
IDLE Shell 3.9.6
File Edit Shell Debug Options Window Help
Python 3.9.6 (tags/v3.9.6:db3bb3b, Sep 6 2021, 12:08:12) on win32
Type "help()", "copyright()", "credits()", or "license()" for more
>>> print("hello world!")
hello world!
>>>
```

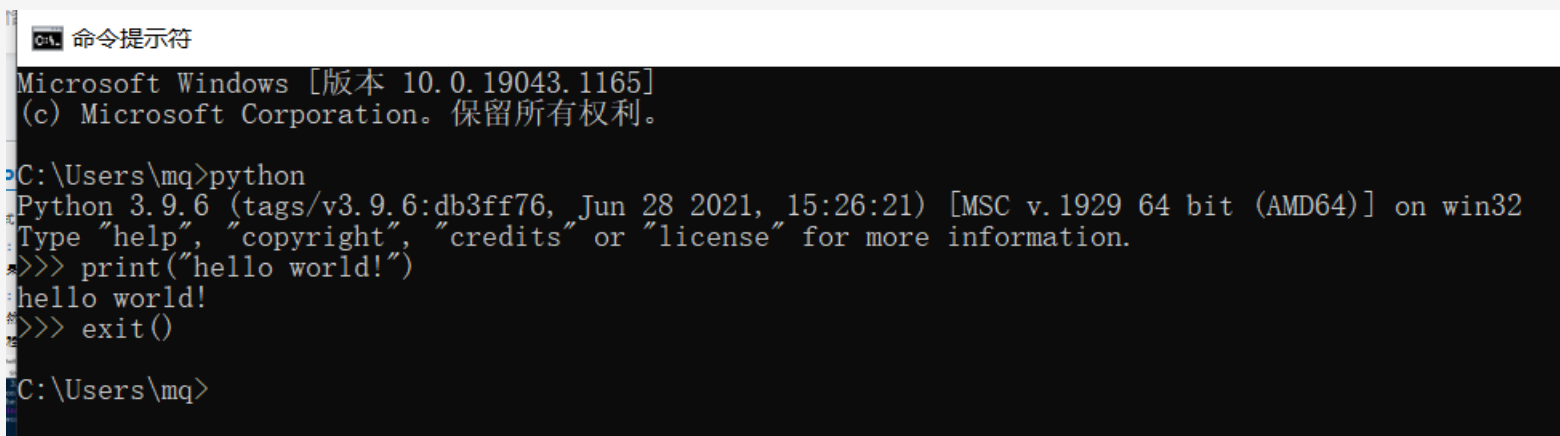


```
MIWAE.py x task01.py x utils.py x
1 import numpy as np
2
3
4 def imputationRMSE(model, Xorg, Xz, X, S, L):
5     """
6     Imputation error of missing data
7     """
8     N = len(X)
9
10    def softmax(x):
11        e_x = np.exp(x - np.max(x, axis=1)[:, None])
12        return e_x / e_x.sum(axis=1)[:, None]
13
14    def imp(model, xz, s, L):
15        l_out, log_p_x_given_z, log_p_z, log_q_z = model.l_out_mu, model.log_p_x_give
16        {model.x_pl: xz, model.s_pl: s, model.l_out_mu: l_out, log_p_x_given_z: log_p_x_given_z, log_p_z: log_p_z, log_q_z: log_q_z}
17
18
```

Python运行

■ 交互式：

- 方法1：启动Windows命令行工具cmd，输入python，执行语句

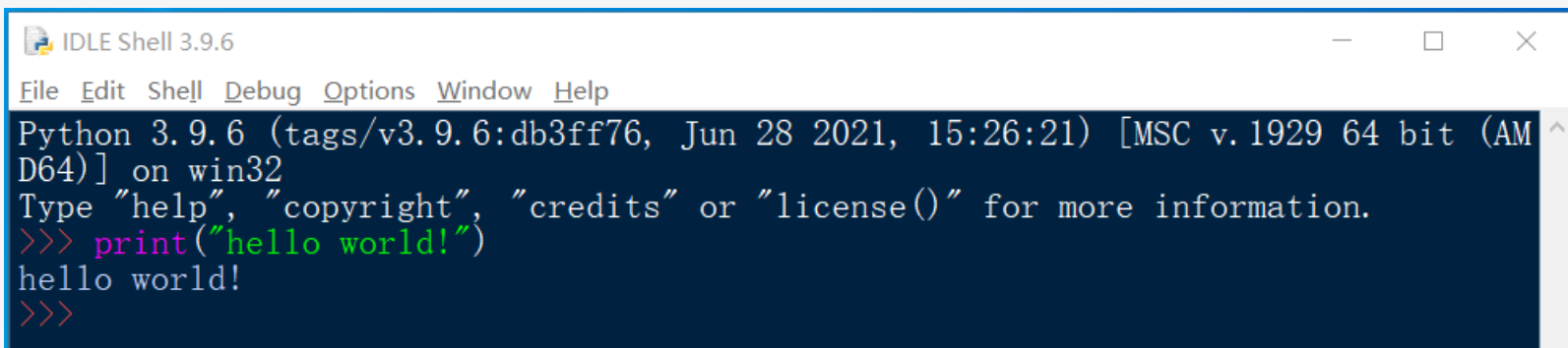
A screenshot of a Windows Command Prompt window titled "命令提示符". The window shows the output of running 'python' at the command line. The text displayed is: "Microsoft Windows [版本 10.0.19043.1165] (c) Microsoft Corporation. 保留所有权利. C:\Users\mq>python Python 3.9.6 (tags/v3.9.6:db3ff76, Jun 28 2021, 15:26:21) [MSC v.1929 64 bit (AMD64)] on win32 Type "help", "copyright", "credits" or "license" for more information. >>> print("hello world!") hello world! >>> exit() C:\Users\mq>".

```
命令提示符
Microsoft Windows [版本 10.0.19043.1165]
(c) Microsoft Corporation. 保留所有权利。

C:\Users\mq>python
Python 3.9.6 (tags/v3.9.6:db3ff76, Jun 28 2021, 15:26:21) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print("hello world!")
hello world!
>>> exit()

C:\Users\mq>
```

- 方法2：调用IDLE来启动Python图形化运行环境

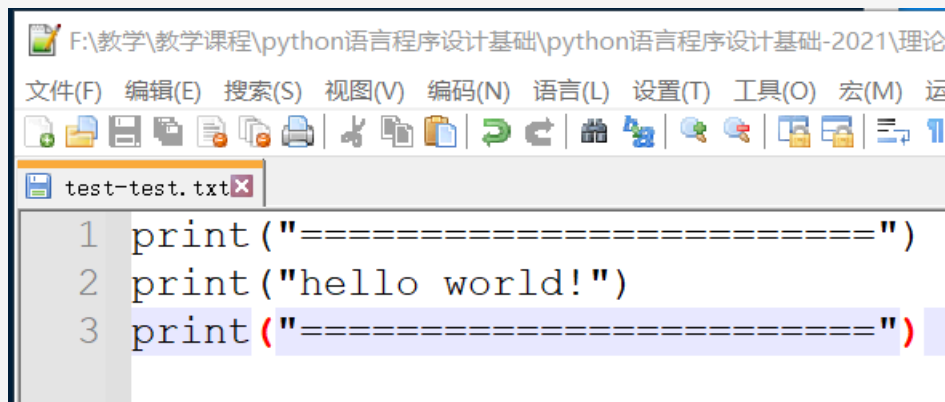
A screenshot of the IDLE Shell 3.9.6 window. The window has a menu bar with "File", "Edit", "Shell", "Debug", "Options", "Window", and "Help". The main area shows the output of running 'python'. The text displayed is: "Python 3.9.6 (tags/v3.9.6:db3ff76, Jun 28 2021, 15:26:21) [MSC v.1929 64 bit (AMD64)] on win32 Type "help", "copyright", "credits" or "license()" for more information. >>> print("hello world!") hello world! >>>".

```
IDLE Shell 3.9.6
File Edit Shell Debug Options Window Help
Python 3.9.6 (tags/v3.9.6:db3ff76, Jun 28 2021, 15:26:21) [MSC v.1929 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> print("hello world!")
hello world!
>>>
```

Python运行

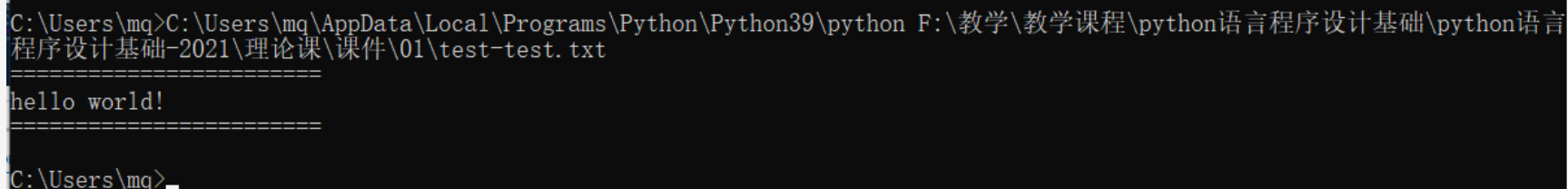
■ 文件式：

- 方法1：按照语法格式编写代码，编写可以用任何文本编辑器，保存为文件，在命令行执行



The screenshot shows a text editor window with the title bar 'F:\教学\教学课程\python语言程序设计基础\python语言程序设计基础-2021\理论'. The menu bar includes '文件(F)', '编辑(E)', '搜索(S)', '视图(V)', '编码(N)', '语言(L)', '设置(T)', '工具(O)', and '宏(M)'. The toolbar contains icons for file operations and editing. The editor has a single tab titled 'test-test.txt'. The code content is as follows:

```
1 print("=====")
2 print("hello world!")
3 print("=====")
```



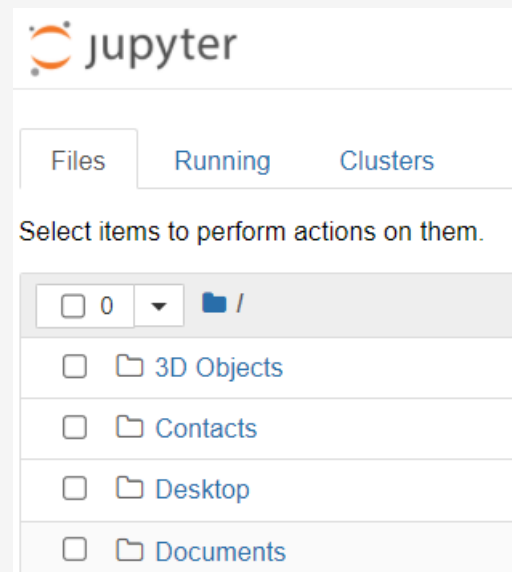
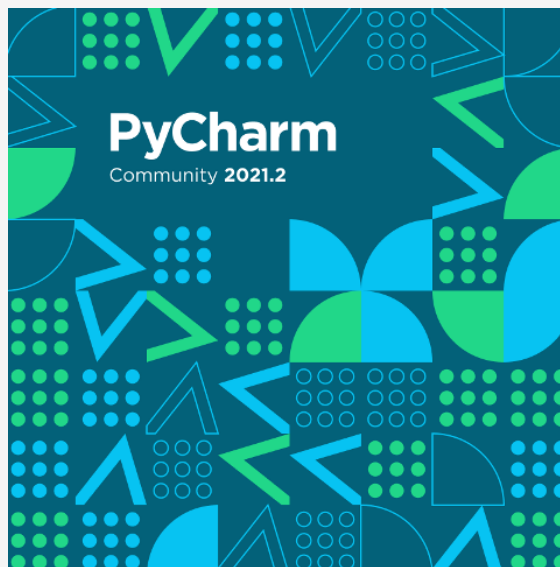
The screenshot shows a command prompt window with the following text:

```
C:\Users\mq>C:\Users\mq\AppData\Local\Programs\Python\Python39\python F:\教学\教学课程\python语言程序设计基础\python语言程序设计基础-2021\理论课\课件\01\test-test.txt
=====
hello world!
=====
C:\Users\mq>
```

Python运行

■ 文件式：

- 方法2：打开IDLE，点击Ctrl+N打开一个新窗口，输入语句并保存，使用快捷键F5即可运行该程序
- 方法3：将Python集成到Eclipse(pydev)、PyCharm、Anaconda、Jupyter等面向较大规模项目开发的集成开发环境中





Python运行

■ 小程序实例：

File Edit Format Run Options Window Help

```
# 圆面积计算
```

```
radius = 25
```

```
area = 3.14 * radius * radius
```

```
print("area: ", area)
```

```
# 简单的人名对话
```

```
name = input("输入姓名: ")
```

```
print("{}同学, 学好python, 前途无量!".format(name))
```

```
# 斐波那契数列的计算
```

```
# 根据斐波那契数列的定义, 计算不大于1000的序列元素
```

```
a, b = 0, 1
```

```
while a < 1000:
```

```
    print(a, end=', ')
```

```
    a, b = b, a+b
```

```
print('\n')
```

```
# 日期和时间输出
```

```
from datetime import datetime # 引用datetime库
```

```
now = datetime.now()
```

```
print("当前时间为: ", now)
```

```
print(now.strftime("%x")) #输出其中的日期部分
```

```
print(now.strftime("%X")) #输出其中的时间部分
```

```
area: 1962.5
输入姓名: Alice
Alice同学, 学好python, 前途无量!
0
1
1
2
3
5
8
13
21
34
55
89
144
233
377
610
987
当前时间为: 2021-08-27 14:14:35.836664
08/27/21
14:14:35
>>> |
```

02

PART TWO

Python版本迭代

Python Version Upgrade

Python版本迭代

- 2010年，python 2.x 已停止维护和更新
 - 目前已发布最新版本是2.7.16
 - 没有python2.8
- 2008年，Python 3.0 (第一个3.x的主版本)发布
 - 目前发布的最新版本是3.12.5
- Python3.x在语法层面和解释器内部做了很多重大改进，解释器内部采用完全**面向对象**的方式实现，版本更迭带来大量库函数的升级替换，几万个函数库进行了痛苦且漫长的版本升级



Python 3.X VS 2.X

- Python 3.x版本不完全向下兼容2.x系列程序
- 语法层面，3.x系列继承了2.x系列绝大多数的语法表达，但移除了部分混淆的表达方式
 - 修改编码：3.X默认采用UTF-8编码，处理中文和英文一样方便
 - 修改print语句：用print()函数代替了print语句
 - 修改exec语句：用exec()函数替换了exec语句
 - 去掉<>符号：用!=表示“不等于”
 -